TKDB | TURNKEY DATABASE

ClickHouse

Same SQL, a different engine

What makes ClickHouse fast on billions of rows, how it’s built, and how it scales out across a cluster.

Scott Beall · Turnkey Database (TKDB) · [Event name] · [Date]

TWO KINDS OF DATABASE WORK

OLTP runs the business, OLAP explains it

OLTP · transactions

```
UPDATE orders SET status = 'shipped'
WHERE order_id = 42;
```

- Touches a few rows, all columns
- Thousands of small writes per second
- Row store, normalized, full ACID

PostgreSQL · MySQL · MariaDB · Oracle · SQL Server

OLAP · analytics

```
SELECT pickup_zone, avg(fare_amount)
FROM trips GROUP BY pickup_zone;
```

- Scans billions of rows, a few columns
- Bulk loads, mostly appends
- Column store, wide tables

ClickHouse · Snowflake · BigQuery · Redshift · Druid · DuckDB

CLICKHOUSE IN ONE SLIDE

ClickHouse: an open-source, column-oriented SQL database for analytics

Built for big scans

Billions of rows, answers in seconds or less.

Plain SQL

Joins, window functions, and speaks the MySQL and Postgres protocols.

Open source

Apache 2.0, one C++ binary. Self-host it or use ClickHouse Cloud.

Laptop to cluster

Same engine on one machine or hundreds of servers.

Where it came from

Built at Yandex for web analytics, open-sourced in 2016.

What people use it for

- Real-time dashboards
- Logs, metrics and traces
- Product and web analytics
- Reporting over event history

STORAGE LAYOUT

Reading rows is slow. Reading columns is fast.

```
SELECT avg(fare_amount) FROM trips WHERE passenger_count = 2
```

Postgres reads rows

SLOWER

RELATIONAL: POSTGRES, MARIADB, ORACLE



Every row is stored whole, so to get 2 columns the database still pulls **all 6 off disk**.

ClickHouse reads columns

FASTER

COLUMNAR: BUILT FOR ANALYTICS



Every column is its own file, so it reads **only 2 of 6** and skips the rest (grey).

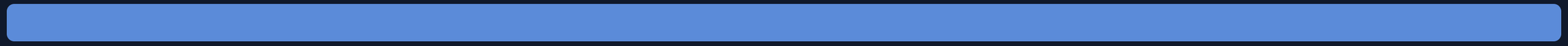
Less data read means a faster query. Real tables have 50 to 200 columns; a report uses a few.

COMPRESSION

ClickHouse: store less, query faster

Analytic queries spend most of their time moving bytes from disk into memory. ClickHouse compresses every column, so there are far fewer bytes to move and the scan finishes sooner.

Relational (Postgres): 1 TB of rows, mostly uncompressed, about 8 minutes to read at 2 GB/s



ClickHouse: the same data compressed 10× to 100 GB, about 50 seconds on the same disk



Illustration: real ratios depend on your data.

Why ClickHouse compresses better

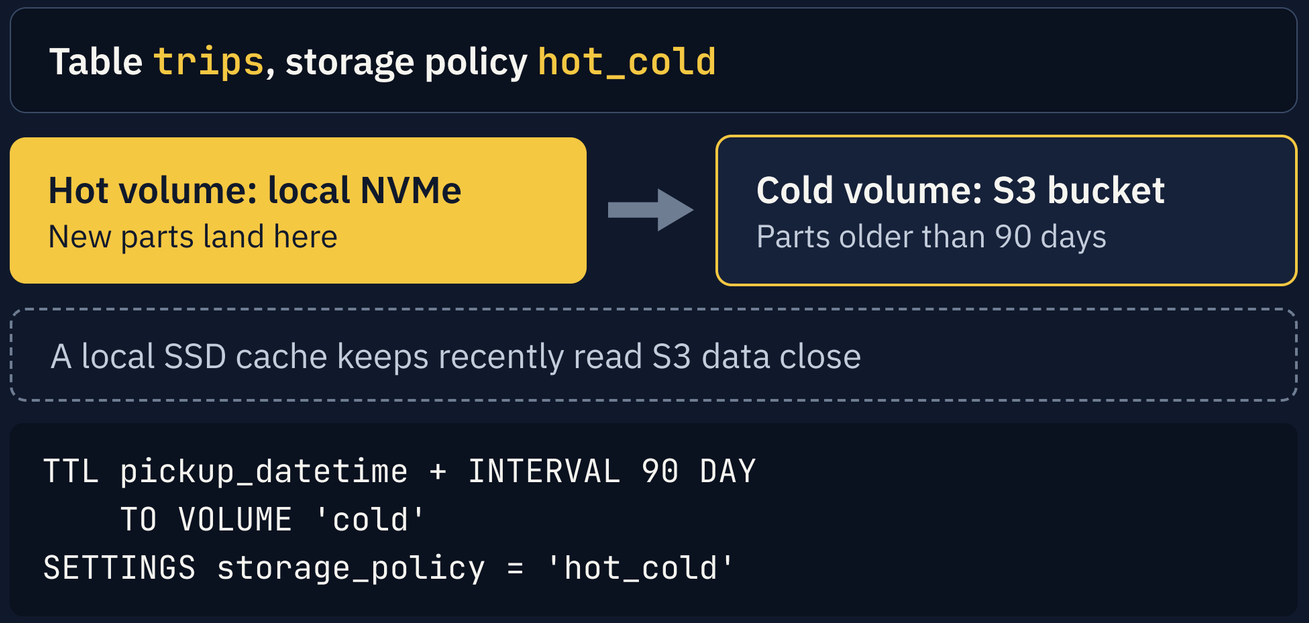
A Postgres page mixes numbers, text and dates from whole rows. A ClickHouse file holds one sorted column of one type, so values repeat.

A codec for each column

DoubleDelta for timestamps, Gorilla for floats, LowCardinality for repeated strings, then LZ4 or ZSTD.

STORAGE

One table, local disks and S3 together



Local disks

NVMe, SSD or HDD. Spread one volume across several disks.

Object storage

S3, GCS or Azure Blob. Cheap, and it never fills up.

Same SQL

Queries don't change when parts move. No archive table, no ETL job.

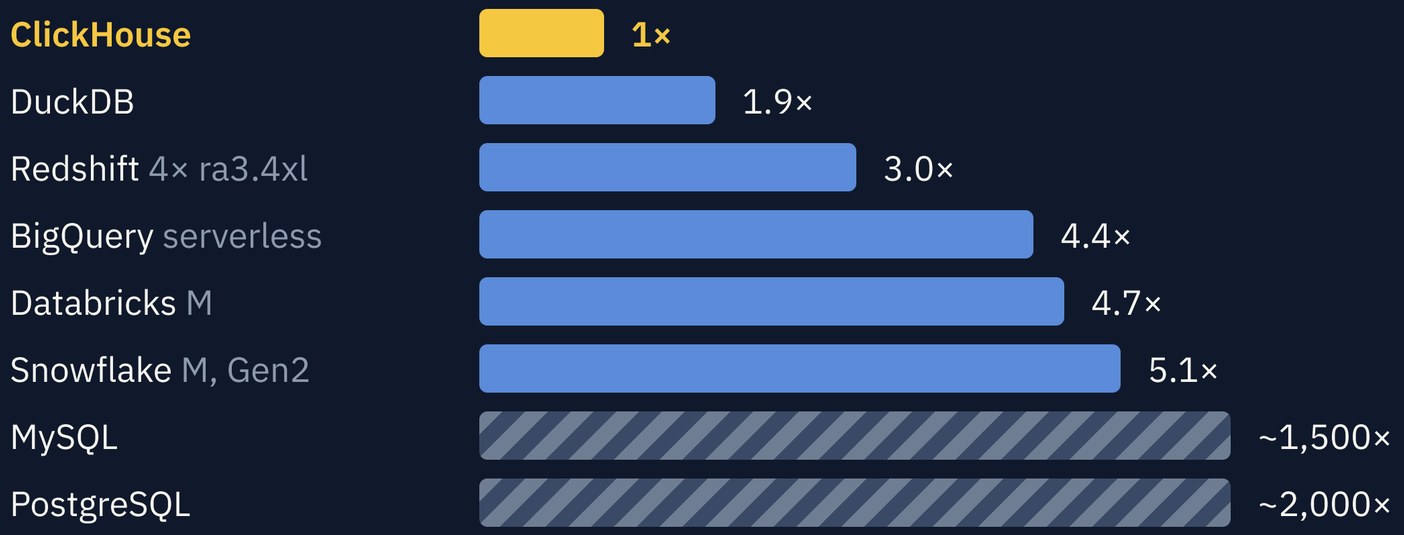
Open data lakes

Query Iceberg tables in S3 in place, shared with Spark or Snowflake.

HOW FAST, COMPARED TO WHAT?

ClickBench: 43 queries, 100 million rows

Time relative to ClickHouse (1×), hot runs. Shorter is faster.



Striped bars are off the scale. ClickHouse, DuckDB, MySQL and PostgreSQL ran on one 16-vCPU server.

Read it fairly

Run by ClickHouse Inc.
But queries, scripts and results are open on GitHub.

One workload
One wide table, few joins. For joins, look at TPC-H or TPC-DS.

Different hardware
Cloud services ran at the sizes shown. Compare cost per query too.

Source: ClickBench (benchmark.clickhouse.com), result files on GitHub as of 17 Oct 2026. Redshift result from 2022.

SIDE BY SIDE

ClickHouse vs relational

Relational (Postgres, MariaDB)	ClickHouse
B-tree index	ORDER BY key (sparse primary index), plus skip indexes
UNIQUE and primary key constraints	Keys sort, they don't block duplicates; ReplacingMergeTree removes them
Table partitioning	PARTITION BY, and drop a whole month in one command
Trigger or summary table	Materialized view into a Summing or Aggregating table
UPDATE and DELETE	Lightweight DELETE, ALTER … UPDATE, or ReplacingMergeTree
Single-row INSERTs	Batches of thousands of rows, or async_insert
Normalize, then join	Joins work; wide tables and dictionaries are faster
Multi-statement transactions	Each INSERT is atomic; no cross-table ACID
One primary, read-only replicas	Every replica takes reads and writes; Keeper keeps them in sync

PERFORMANCE FEATURES

Features that make queries faster

Materialized views

Aggregate on every insert. Dashboards read thousands of rows, not billions.

```
CREATE MATERIALIZED VIEW … TO
```

Refreshable views

Rebuild a heavy result on a schedule, like REFRESH in Postgres.

```
REFRESH EVERY 1 HOUR
```

Projections

A second sort order or rollup inside the table. The optimizer picks it.

```
ADD PROJECTION p (SELECT …)
```

Skip indexes

Skip whole granules on columns outside the sort key.

```
TYPE minmax | bloom_filter
```

Dictionaries

Small lookup tables held in memory, in place of a join.

```
dictGet('zones', 'borough', id)
```

Query cache

Reuse results when the same dashboard query repeats.

```
SETTINGS use_query_cache = 1
```

Also working for you: PREWHERE reads the filter column first, and async_insert batches small writes.

TABLE ENGINES

Every table picks an engine

A table engine is the storage type you pick for each table. It decides where the data is stored, how it is indexed, what happens on insert and merge, and whether it is replicated.

```
CREATE TABLE trips (…)
ENGINE = MergeTree
ORDER BY (pickup_zone, pickup_datetime)
```

MergeTree family

Where your data lives. Each variant changes what happens when parts merge.

`MergeTree` · `Replacing` · `Summing`
`Aggregating` · `Collapsing` · `Replicated*`

Integrations

Read or write other systems with plain SQL, without copying first.

`PostgreSQL` · `MySQL` · `MongoDB` · `Kafka`
`S3` · `Iceberg` · `DeltaLake` · `URL`

Special purpose

Plumbing: fan out to shards, look up, buffer or discard writes.

`Distributed` · `Dictionary` · `Merge`
`Buffer` · `Null` · `Memory` · `Join` · `Set`

Log family

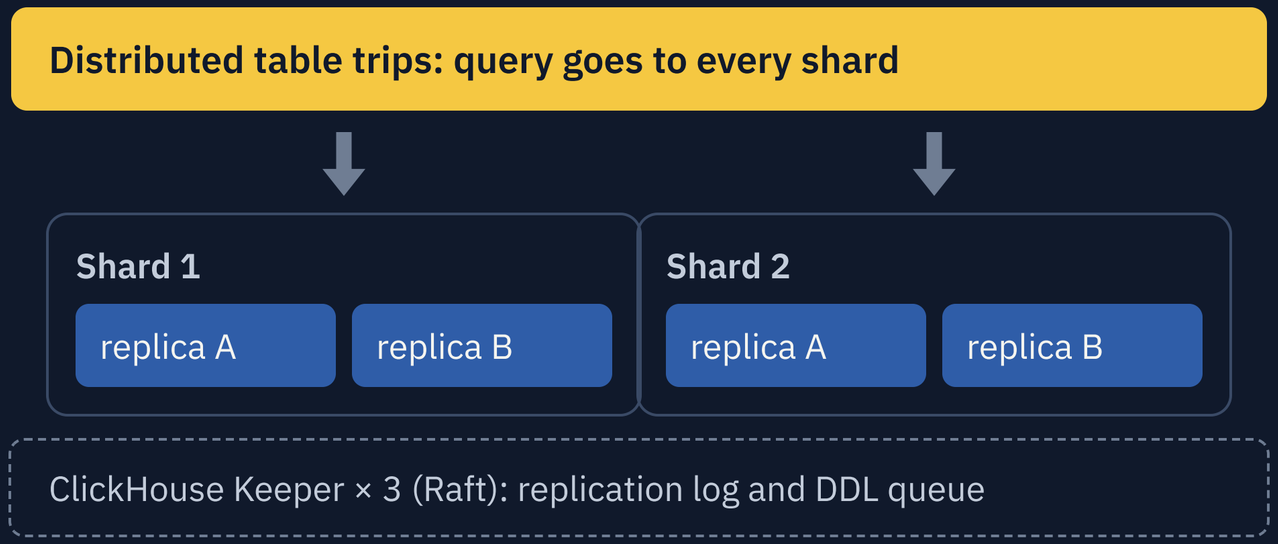
Simple, write-once tables for small or temporary data.

`TinyLog` · `StripeLog` · `Log`

CLUSTERING

Shards for size, replicas for safety

Sharding splits one large table across several servers, each holding a slice of the rows. Every shard works on its slice at the same time, so adding servers adds disk, memory and CPU, and big queries finish faster.



Need more capacity?

Add a shard. Each holds a slice of the rows.

Need more safety?

Add a replica. ReplicatedMergeTree copies parts between peers, and any replica accepts writes.

One DDL for all

ON CLUSTER runs a CREATE or ALTER on every node.

OLTP AND OLAP TOGETHER

Keep Postgres and other relational databases, and add ClickHouse beside it



Query it in place

The postgresql() and mysql() table functions read live tables, joinable with ClickHouse data.

Replicate it

CDC streams changes into ReplacingMergeTree tables, close to real time.

Look it up

Dictionaries cache small Postgres or MariaDB tables in memory for fast joins.

Best of both worlds: transactions stay safe in the relational database, analytics run fast in ClickHouse, and neither slows the other down.

WHY SWITCH

Lower cost, a simpler stack, and it runs anywhere

Cheaper than Redshift

~60×

lower compute cost per query on ClickBench

Redshift, 4 × ra3.4xlarge at $13.04/hr, took 3× as long as ClickHouse on one c6a.4xlarge at $0.61/hr.

Replaces legacy Accumulo

- No Hadoop, HDFS or ZooKeeper cluster to run
- SQL instead of custom Java iterators
- Row policies and grants for fine-grained access

Runs where you need it

TKDB delivers it Turnkey:

- **On-prem:** one binary on bare metal or VMs, even air-gapped
- **Amazon EKS:** Kubernetes operator and Helm, EBS hot, S3 cold
- **Managed:** TKDB-managed Kubernetes cluster

Cost: ClickBench hot-run times × AWS on-demand list prices, us-east-1, compute only. Redshift result from 2022.

DEPLOYING WITH TKDB

ClickHouse on Kubernetes

Kubernetes is a platform that runs software across a group of servers. You describe what you want, and Kubernetes keeps it that way: if a server dies, it restarts the work somewhere else.

Runs anywhere: on-prem in your own data center, or on AWS with Amazon EKS. Same files, same operator.

You declare it:

```
kind: ClickHouseInstallation
spec:
  configuration:
    clusters:
      - name: main
        layout:
          shardsCount: 2
          replicasCount: 2
```

Pod
One running server process, here a ClickHouse node.

StatefulSet
Database servers with fixed names and their own disks.

Persistent volume
The data disk: EBS on EKS, local disks on-prem.

Operator
A DBA runbook as code: builds, scales and upgrades the cluster.

HELM · VALUES.YAML

The whole database in one short file

Helm is the package manager for Kubernetes: one install, one settings file, the whole cluster.

```yaml
layout:
  shardsCount: 2   # split data across 2
  replicasCount: 2 # 2 copies of each
image:
  tag: "24.8"      # ClickHouse version
resources:
  requests: { cpu: 250m, memory: 512Mi }
  limits:   { memory: 2Gi }
storage:
  data: { size: 2Gi } # disk per server
```

Chart

A versioned package for the whole database.

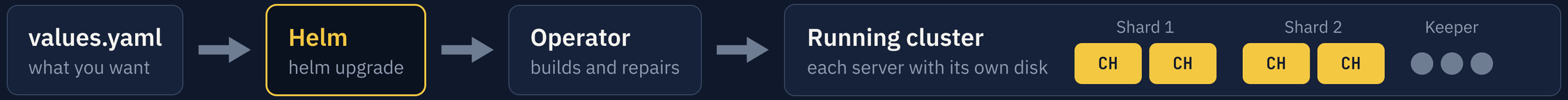
Values

This file: size, version, memory, disks. Kept in Git and reviewed like code.

Release

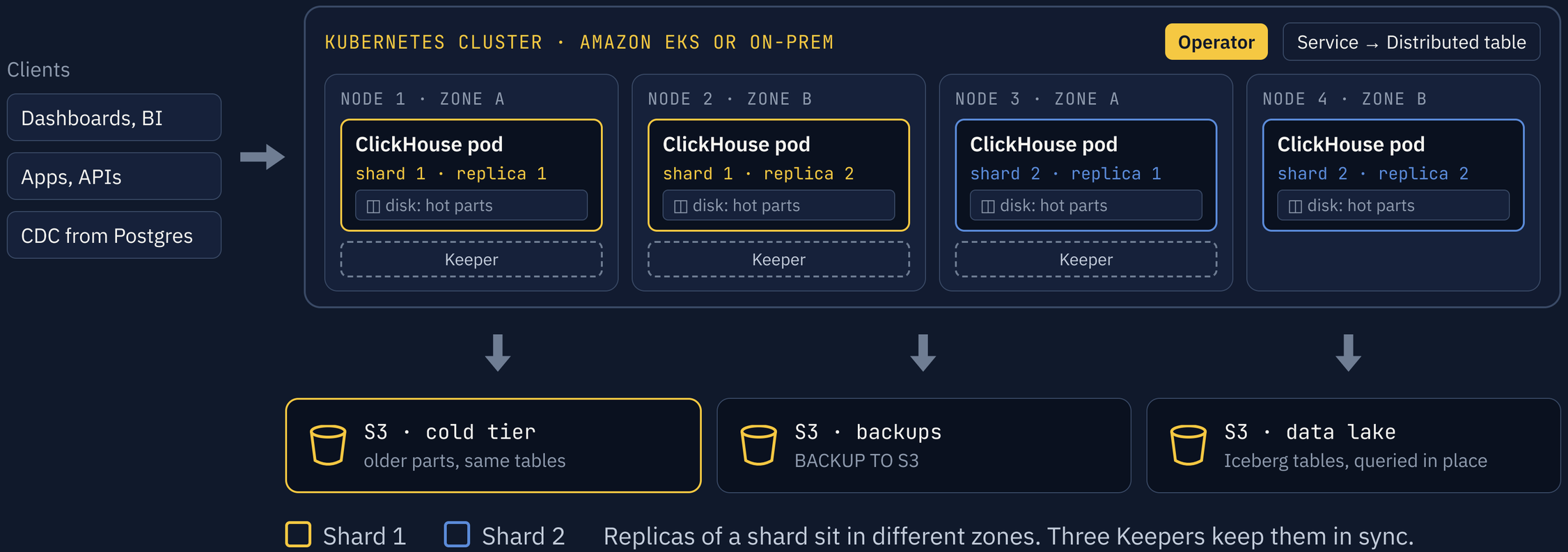
Every change is numbered, so you can roll back.

`helm install` `helm upgrade` `helm rollback`



THE BIG PICTURE

One ClickHouse cluster on Kubernetes, end to end



WHAT TKDB DOES

Fast analytics, Turnkey, wherever your data has to live

TKDB takes you from a slow, costly analytics stack to ClickHouse running in your own account or data center, and then runs it for you.

STEP 1

Assess

Prove it on your own queries

- Run your top queries on ClickHouse
- Cost per query vs Redshift or Snowflake
- A clear go or no-go

STEP 2

Migrate

Move with a fixed scope

- Accumulo and Hadoop
- Redshift and Snowflake
- Postgres and MySQL reporting via CDC

STEP 3

Run

A TKDB-managed Kubernetes cluster

- In your AWS account (EKS) or on-prem
- Upgrades, scaling, backups, monitoring
- Your team just writes SQL

Your data stays yours
Your account, your network, your keys

On-prem and air-gapped
Offline install, hardened images

Open source, no lock-in
Apache 2.0 ClickHouse, open Helm chart

Speaks DBA
Relational habits mapped to ClickHouse

TAKEAWAYS

1 **Right tool:** OLTP runs the business, ClickHouse explains it.

2 **Read less:** columns and compression shrink every scan.

3 **Store smart:** hot data on local disk, history on S3, one table.

4 **Think differently:** engines, batches and duplicates work their own way.

5 **Scale out:** shards for size, replicas for safety, Postgres alongside.

Questions? · Scott Beall · Turnkey Database (TKDB)